Škoda Storyboard and Media Room on Adobe Edge Delivery Services

What we built, and why

Date: October 15, 2026

Adobe

Table of Contents

About this document 3
Executive summary 4
Where we started 5
The current platform 5
What makes it hard 5
How we analysed it 5
The platform we chose 6
Edge Delivery Services with Document Authoring 6
How the system works 7
Content: documents, sections and blocks 7
Front end: rebuilt, not ported 7
Listings, rails and search 7
Media and AEM Assets 8
The media cart 8
Navigation, footer and languages 9
Links between the demo and the live site 9
The authoring experience 9
How content is migrated 10
How we checked quality 11
What is live today 12
What we deliberately did not do yet, and why 13
Things to be aware of 14
The road to go-live (2 January 2027) 15
Glossary 16

About this document

This document explains how the new Škoda Storyboard and Media Room platform was built for the 15 October 2026 demo milestone. It covers what we did and why we did it. It is written for business, editorial and product stakeholders. Technical detail is kept to what you need to follow the decisions.

It covers:

- the starting point: what the current site is and what makes it hard to migrate;
- the platform we chose and why;
- how the new system works: content, design, listings, media, the media cart, languages and authoring;
- how content is moved from the current site to the new one;
- how we checked quality against the original site;
- what is live today, what we deliberately left out, and what remains for go-live on 2 January 2027.

Executive summary

Škoda Storyboard is Škoda Auto's global editorial site and press newsroom. It runs on a large, heavily customised WordPress installation with six languages, an Elasticsearch-based search and listing layer, a large media library, and several custom backend features.

We are rebuilding it on **Adobe Experience Manager Edge Delivery Services**. Content is authored in **Document Authoring (Experience Workspace)** and approved media comes from **AEM Assets**. The 15 October demo had four client goals: a working front end, approved AEM Assets, replicated blog posts and a replicated media cart. All four are in place on the English site. Other languages follow in the go-live phase.

What makes this a real system and not a one-off copy:

- **A clean rebuild of the front end.** The site was not copied over. Each page component was rebuilt in plain, modern code that loads quickly, measured against the original site.
- **Automated, repeatable content migration.** Pages are converted by import scripts, one per page type, and pass through safety checks before they are published. The same scripts will carry the full content migration for go-live.
- **Listings and rails that fill themselves.** Story feeds, category rails and media listings read a published index of the site. Editors publish a page and it appears in the right places, with no manual curation.
- **Media from a single source of truth.** Images are served in web-optimised formats. Press downloads (originals, PDFs, videos) come from AEM Assets.
- **A media cart with no login.** Journalists collect assets across pages and download them as one zip file. Nothing is stored about them on a server.

As of today, **313 pages are published** on the live demo site. A Lighthouse test of a representative story page scored **99** for desktop performance.

Where we started

The current platform

Aspect	What we found
Site type	Editorial site and press newsroom (no e-commerce)
Technology	WordPress, server-rendered, with jQuery and many custom plug-ins
Languages	6 (Czech, Slovak, English, German, Serbian, Slovenian), translated unevenly per article
Editorial pages	About 13,300 across all languages, about 3,500 of them in English
Media	About 42,000 attachment pages, about 28,000 distinct items, about 200,000 files once all the generated image sizes are counted
Search and listings	Elasticsearch (ElasticPress) with 15 filter groups

What makes it hard

We identified three things that shape the whole migration:

1. **Scale.** Thousands of pages in six languages, and a media library measured in tens of gigabytes.
2. **Backend features that are not just pages.** Search and listings, a per-market banner ad system, a newsletter subscriber system, the media cart and a custom analytics layer all depend on server logic. Each needs an explicit decision: rebuild it as a service, simplify it, or drop it.
3. **Front-end debt.** The current front end relies on jQuery, several heavy carousel and layout libraries, a 174 KB React bundle for banners and a 295 KB stylesheet with 30 different breakpoints. Moving this code across would carry the slowness and the maintenance cost into the new platform.

There was also good news. The content model is consistent: a small number of page types reuse about ten content components. Most of the responsive behaviour is pure CSS, so it ports cleanly. The header and footer turned out to be simpler than they first appeared.

How we analysed it

Before building, we crawled all 3,496 English pages, inspected the WordPress REST API and sitemaps, measured the media library, and recorded the behaviour of every complex feature using read-only requests. We also measured every component in a browser at the site's main breakpoints (768, 992 and 1080 pixels, plus mobile), so we had real numbers to build against. Two internal red-team reviews then challenged our findings and corrected several early estimates. For example, the media count fell from an assumed 52,000 assets to a measured 28,000 distinct items.

Why: A migration of this size fails on surprises. Measuring first turned most of the unknowns into known quantities before we committed to a design. It also let us size the work from evidence instead of assumptions.

The platform we chose

Edge Delivery Services with Document Authoring

Layer	Choice
Authoring	Document Authoring / Experience Workspace (da.live)
Approved media	AEM Assets
Code	A GitHub repository with no build step, synchronised automatically
Delivery	Adobe's edge network: a preview site (aem.page) and a live site (aem.live)

Why Edge Delivery Services:

- **Performance by design.** Pages are delivered as static HTML from the edge, and scripts load in three phases (essential, then visible, then everything else). This setup reliably scores at or near 100 in Lighthouse, which helps search ranking and the experience on mobile networks.
- **Search engines see real content.** On the current site many listings load through browser requests after the page opens. On the new site the content is in the page itself.
- **Low operating cost.** There are no application servers or databases to patch and scale for the public site.
- **Fast, safe change.** Every code change gets its own preview link, and content and code publish independently.

Why Document Authoring (Experience Workspace):

- Editors work in a familiar document editor with a visual layout mode, an AI assistant, bulk publishing and built-in translation.
- It fits a content-heavy editorial site where most pages follow a few repeatable patterns.
- It can be extended with custom authoring tools without extra infrastructure (see "The authoring experience" below).

The trade-off we accepted: Document Authoring cannot index content from spreadsheets. All listings, feeds and search therefore use an index built from published pages. This turned out to be a strength, because a page appears in feeds as soon as it is published. It does mean that publishing order matters (see "Listings, rails and search").

How the system works

Content: documents, sections and blocks

Each page is a document. A document is divided into **sections**, and each section holds default content (headings, text, images) and **blocks**. A block is a reusable component, such as a hero, a card grid, a gallery or a downloads list. At the end of each page, a metadata table records the title, description, image, template, category, publish date and tags.

Why: Authors work with structured, predictable content instead of free-form page-builder layouts. The same structure lets the site build its indexes, feeds and language links automatically.

Front end: rebuilt, not ported

We built about 25 blocks, covering the patterns used on the site:

- **Page tops:** hero and hero image (including archive and overlay variants).
- **Cards and rails:** cards, carousels, story rails and a promo box (curated or index-driven).
- **Story and press content:** galleries with a lightbox, quotes, columns, accordions, footnotes, tags, embeds (YouTube, Vimeo, Buzzsprout) and downloads (the "Media Box").
- **Listings and search:** a faceted listing for news, images, videos and search; the stories feed.
- **Site chrome:** header with mega-menu and mobile drawer, footer, floating dock (scroll-to-top and cart badge), newsletter card and the media cart page.

Page types whose layout differs from the standard, such as the press release, have their own page template.

All blocks are written in plain JavaScript and CSS, with no frameworks and nothing to compile. Škoda's colours, typography, spacing and card styles are defined once as design tokens, and every component uses them.

Why:

- **Speed.** Removing jQuery, three carousel and layout libraries and the React banner bundle cut most of the code the browser has to download.
- **Maintainability.** One clear set of tokens and components replaces 168 `!important` overrides and 30 breakpoints. The CSS follows written guidelines that an automated linter enforces.
- **Accessibility upgrades.** We fixed known gaps in the original instead of copying them. The mobile menu is now a real button with correct screen-reader states, carousels have controls, the lightbox manages keyboard focus, and the media cart announces changes to screen readers.
- **Editable in place.** Images are optimised without being replaced, so editors can still edit them in Experience Workspace's visual layout mode. The standard approach recreates images and makes them uneditable there, which we found and fixed early.

Listings, rails and search

The homepage feeds, category rails, model page rails, archives, and the news, image and video listings are not hand-built. They read the site's **query index**: a JSON file that Edge Delivery Services rebuilds from published pages. Each block then filters, sorts and pages the results in the browser. The faceted listings offer the same filter groups as the original, plus newest/oldest sorting, "Load more" and shareable URLs.

On the homepage, rails remove duplicates so that a story already shown higher up is not repeated further down.

Why:

- It replaces Elasticsearch for listings with no search server to run.
- Editors never curate feeds by hand. Publishing a page with the right tags puts it in every relevant rail and listing.
- The import step writes the index fields into page metadata, so tags, categories and dates stay consistent. Real source pages did not always carry clean dates or categories, so the importer derives them from several fallbacks. A validation step blocks any page with missing index fields.

What to be aware of:

- The index only contains **published** pages. A page that is previewed but not yet published does not appear in feeds.
- Index-based search matches titles, summaries and tags. It does not offer Elasticsearch's full-text, typo-tolerant ranking. That would need a hosted search service, which is a production decision for go-live.

Media and AEM Assets

- **Originals stay in AEM Assets.** AEM Assets is the approved source for press-quality originals, PDFs and MP4 videos.
- **Web images are optimised automatically.** On publish, Edge Delivery Services stores each page image and serves it in modern formats (WebP) at the right size for each screen. We import only master images and leave out the roughly 200,000-file ladder of pre-generated sizes on the current site.
- **Oversized images are handled before publishing.** Some source masters are 25–40 MB, which the publishing service rejects. The import pipeline swaps them for a verified web-sized version first, so pages publish reliably.
- **A media manifest tracks every file.** Each migrated image, PDF and video has a record of where it came from and where it is delivered from now. This record is kept with the import.
- **Image and video listings run from a generated feed** built from the source listing data. This avoids creating thousands of single-asset pages.
- **Captions and alt text are kept.** About half the source captions were hidden in a data attribute, and the importer recovers them.

Why: Keeping originals in AEM Assets gives editors and journalists one source of truth for approved media. It also gives the media cart files it can fetch. Leaving out the generated size ladder cuts the migrated footprint from an estimated 80–110 GB to the masters, while the new platform generates better formats on demand.

The media cart

The cart works the way the client specified: **no login, tied to the device**.

- Visitors add images, videos and documents from any page. The cart is stored in the browser and survives page changes and return visits. A badge on the floating dock shows the number of items.
- The cart page groups items by type and shows each item's format and size.
- A single item downloads directly. Several items are fetched from AEM Assets and zipped **in the browser** into one download, with progress shown and any failed files listed.

Why: Edge Delivery Services has no server in the delivery path that could build zip files on request. The browser can do it, and AEM Assets delivery allows the cross-site file access this needs (the current CDN does not). The result needs no new backend, stores no personal data, and delivers press-quality originals instead of web renditions. Production hardening, such as very large selections, signed access and cross-device carts, is planned separately for go-live.

Navigation, footer and languages

- The header, mega-menu and footer are **shared fragments**: one document for each, edited once and used on every page. The Media Room variants are handled through page metadata.
- The language switcher only offers languages a page actually exists in, as the client required. This comes from language-link metadata generated during import, which is also what search engines read.
- The published demo is in English. The structure is ready for other languages: each language gets its own content tree, its own navigation and footer, and its own index. Czech comes first, then German, Slovak, Serbian and Slovenian on demand per local office, using Experience Workspace's translation tools.

Why: Shared fragments mean the navigation is maintained in one place. Language links built from real page availability prevent links to translations that don't exist.

Links between the demo and the live site

The demo carries only part of the site, so every link follows a clear rule. A link to a page we have migrated stays on the new site. A link to a page we have not migrated goes to the current live site. Links are normalised to the new platform's format (for example, with no trailing slash), and a dead-link report checks the whole site.

Why: Visitors never land on a missing page during the transition, and the rule is automatic.

The authoring experience

We reviewed the authoring experience against Adobe's best practice. As a result, we added:

- **A block library inside the editor**, with ready-to-insert examples of every block and variant.
- **A tag picker plugin**, so authors choose tags from the agreed list instead of typing them. This works without extra infrastructure.
- **AEM Assets in the editor**, so authors pick approved images directly.
- **Shared fragments for repeated content.** For example, the newsletter card on every story is one document, edited once.

Why: A migration is only successful if editors can run the site after it. We designed every block so that its content is either edited directly on the page or set through a small, clearly labelled configuration table.

How content is migrated

We did not move content by hand. Each page type has its own **import script**: the homepage, stories, press releases, press kits, model pages, series hubs, archives, listings, Škodapedia and company pages, the 404 page and the navigation. The script reads a page from the current site and produces a clean document in the new structure.

The process for every batch of pages:

1. **Import.** Convert the source pages into structured documents.
2. **Prepare media.** Record every image and file, pick a publish-safe version of each image, and point downloads at AEM Assets.
3. **Validate.** Check that metadata is complete, that every block used exists, and that no document breaks platform limits (for example, a maximum of 200 images per document; larger press-kit galleries are split into fragments automatically).
4. **Push to Document Authoring.** The push tool records what it last uploaded. If an editor has changed a page since then, it **refuses to overwrite it** unless someone confirms.
5. **Preview and review.** The preview site acts as the draft.
6. **Publish and check.** Pages are published only after they pass validation. Shared fragments (navigation, footer) must be live first. The tool then confirms each page has appeared in the index.

Why:

- **Repeatable.** When an importer improves, we re-run it across all pages of that type instead of fixing pages one by one.
- **Scales to go-live.** The same scripts and checks will migrate the remaining content.
- **Safe.** Editor changes are never silently overwritten, broken pages are held back, and every run leaves a written report.

How we checked quality

- **Measured against the original site.** Every page type and component was compared with the live original in a real browser at the agreed breakpoints. We compared measured values (positions, sizes, spacing, fonts, colours, structure), not opinions from screenshots.
- **Separate building and checking.** The work ran as a loop: plan, build, then an independent quality check. Anything that failed went back for another round until it passed.
- **Automated tests and linting** for blocks, import scripts and the link rules.
- **Performance testing.** A representative story page scored 99 for desktop Lighthouse performance, with no main-thread blocking. Heavy embeds like YouTube load only when clicked.
- **Traceability.** Every client requirement ID is mapped to a ticket and a build status, so it is clear what is done, in progress or still open.

What is live today

As of 15 October 2026 the English demo includes:

Area	Status
Storyboard homepage	Live: curated promo, latest stories, category rails, models rail, series and media rails
Stories	Live, including galleries, embeds, quotes and the Media Box
Press releases	Live, with their own layout, related items and downloads
Press kits	Live: hubs with chapter navigation, chapter and resource pages, ZIP links (including the Classic Days 2026 kit)
Model pages	Live: hero, key facts and model-specific rails
Series, category and tag archives	Live
Media Room homepage	Live: press release promo from the index, news, images, videos, models, press kits and stories rails
News, Images, Videos listings	Live, with filters, sorting and “Load more”
Media cart	Live: collect across pages, review, download originals as a zip
Header, footer, mobile navigation, language switcher	Live
Authoring tools	Block library, tag picker and AEM Assets picker configured

In total, **313 pages are published** on the live site, and new content from the current site, such as the October Epiq Sportline story and press release, has been imported with the same pipeline.

What we deliberately did not do yet, and why

Item	Decision	Reason
Banner ad platform	Not rebuilt for the demo	It is a custom per-market ad server. Worth confirming whether a simpler promo block meets the need before rebuilding it.
Newsletter subscriptions	Card and links only	The subscriber system needs a decision on the email provider and consent model.
Custom analytics layer	Not rebuilt	It depends on Škoda's analytics specification. Every interactive block is ready to emit events once it is confirmed.
Full-text search	Index-based search for the demo	Typo-tolerant, ranked search needs a hosted search service. This is a production decision.
Content embargoes	Not built	This is a go-live requirement. We recommend a staged-publish approach, because the platform has no built-in time gate.
Parallax effects	Dropped	Decorative only, and costly for performance.
Pre-generated image sizes	Not migrated	The new platform generates better formats on demand.

Things to be aware of

- **Publishing order matters.** Pages appear in feeds only once they are **published**. Shared navigation and footer must be published before the pages that use them, or the live site shows them empty.
- **Domain-locked videos.** Some Vimeo videos only allow playback on the current site's domain. Their privacy settings must add the new domain before go-live.
- **Search scope.** Until a hosted search is chosen, search covers titles, summaries and tags, not full article text.

The road to go-live (2 January 2027)

The demo proves the full system end to end on English content. Go-live builds on the same foundations:

- migrate the remaining content with the existing import scripts: the long tail of English stories, then Czech, then the other languages on demand;
- finish the remaining "Page" sub-types (board of management, annual reports, logos, contacts);
- decide and implement the open backend items: banners, newsletter, analytics and hosted search;
- add authoring governance: roles, content embargoes, audit and QR-restricted access;
- harden the media cart for production use;
- finalise redirects from current URLs to new ones.

Glossary

Block A reusable page component, such as a hero, cards or a gallery, that authors place in a document.

Document Authoring / Experience Workspace Adobe's document-based editor where authors write and publish pages.

Edge Delivery Services Adobe's delivery platform. It serves pages as fast static HTML from a global edge network.

Fragment A document reused on many pages, such as the navigation, footer or newsletter card.

Query index A machine-readable list of published pages and their metadata. Feeds, rails and listings are built from it.

Preview / Live Every page has a preview version (the draft, on aem.page) and a published live version (on aem.live).

AEM Assets Adobe's digital asset management system. It is the approved source for media originals.